

Class and syntax in Ruby

Class is like a blueprint that combines both characteristics and functions. Individual objects can be formed from a class.

Here is an example.

In case of a cuboid, its height, length, and breadth can be considered to be members of the class Cuboid.

```
class Test

  def welcome
    puts "Welcome Fellow Coders to Ruby Programming!"
  end
end

# Now using above class to create objects
obj = Test.new
obj.welcome
```

This is an example of a simple class 'Test'. *obj* is an instance/object of the class *Test*. On the instance/object, calling the method *welcome* welcomes you to Ruby classes and objects. Let us look at the method in detail, in the next class.

Global variables: Variables that can be added anywhere, throughout a program, are called global variables.

- A global variable starts with the special character – dollar sign (\$). For example: *\$radius*, *\$counter*, *\$name* and *\$5*.
- Remember, until declared, the value of a global variable will always be 'nil'.

What follows is an example of how a global variable is used:

```
$glob
puts #$glob
Output:nil
$global = 10
class C
  puts "in a class: #{ $global}"
end
puts "at top-level, $global: #{ $global}"
Output
• in a class: 10
• at top-level, $global: 10
```

Class variables: Variables within a single class structure are called class variables.

- Class variables begin with @@. Example: *@@total = 0*
- It is necessary to initialise class variables before they can be used in method definitions.
- An error will appear if an uninitialised class variable is referenced.
- Class variables are shared between all instances of a class.

Instance variables: Instance variables help to define the characteristics or attributes of objects.

- Instance variables begin with '@' – a special character. For example: *@ins_var*.
- They cannot be accessed directly from the class

definitions, but only within definite objects across varied methods within a class instance.

- Until declared, the value of an instance variable will always remain 'nil'.

In this example, *Employee.new* creates a new object — an instance of the class *Employee*. The instance variable is *@emp_id*.

```
class Employee
  def initialize(emp_id, emp_name)
    @emp_id = emp_id
    @emp_name = emp_name
  end
  def display
    puts "Employee ID and Name: "
    puts @emp_id, @emp_name
  end
end

Employee.new(23434, "Anand").display
Employee.new(343434, "Ram").display
Output:
Employee ID and Name:
23434
Anand
Employee ID and Name:
343434
Ram
```

Local variables: Local variables have a name beginning with an underscore (_) or a lower case letter (from a-z). Local variables have scope inside the following:

- class ... end
- module ... end
- def ... end
- From a block's opening brace to its close brace. Example: *{}: proc{ ... }, loop{ ... }*
- The whole program (except when any one of the above applies)

Shown below is an example of a local variable:

```
name = "Anand"
def method1
  name = "Anand"
  puts "Name within method1: ",name
end
def method2
  name = "Ram"
  puts "Name within method2: ",name
end
method1
method2
method1
puts "Name outside methods: "+name
Output
```